

of chart decomposition and reuse. Most approaches that focus on this task, such as D3 Deconstructor [31] and ChartReuse [21], require the input charts to be created using a specific tool. A more recent work, Mystique [15], designed a tool-agnostic decomposition and reuse pipeline for general SVG charts composed of rectangle-shape marks based on a diverse 150-chart corpus. What lies at the core of Mystique is a bottom-up hierarchical clustering algorithm that determines nested mark groups and their internal spacial relationships presented in a chart. In this section, we use VISANATOMY to form a validation set to evaluate the generalizability of the hierarchical clustering algorithm from Mystique on unseen charts.

To prepare the validation set, we first filter the charts based on the **Element Type** labels, and only keep those having the **Rectangle** type for all **main chart marks**, resulting in 306 charts. Given that Mystique does not consider radial and spiral layouts, we exclude spiral plots and bar charts in the radial layout. We further remove charts that were already included in Mystique’s corpus. The final validation set consists of 248 charts encompassing 15 chart types. We then run Mystique’s hierarchical clustering algorithm using the **main chart marks** of each chart in this validation set as input, and the results are compared with **Hierarchical Grouping** and **Group Layout** labels in VISANATOMY. For the error cases, we perform another round of manual inspection to avoid false negatives as some charts have multiple reasonable grouping structures (e.g., a diverging bar chart) [15].

Results. 217 charts (out of 248) within the validation set are decomposed into their correct grouping structures and corresponding spatial relationships, making an approximate 87.5% accuracy (8% lower compared to the test accuracy reported in Mystique [15]). Examining the 31 error cases, we report three kinds of failure cases that were not discovered by the authors of Mystique [15]:

1. A slice-and-dice treemap (Figure 7(a)), whose overall packing layout was wrongly recognized as a vertical stack layout by Mystique’s decomposition algorithm, leading to incorrect mark groups;
2. Glyph-based charts where the marks within a glyph do not always overlap, e.g., in Figure 7(b), the three lighter gray bars are stacked within each row without overlapping. The decomposition algorithm uses overlapping relationships to detect glyphs and fails in such cases;
3. A bespoke Gantt-calendar chart (Figure 7(c)) where the bar groups are positioned based on data and the bars within each group follow a horizontal stack layout. The algorithm fails to decompose this chart correctly, giving a lowest-level packing layout instead.

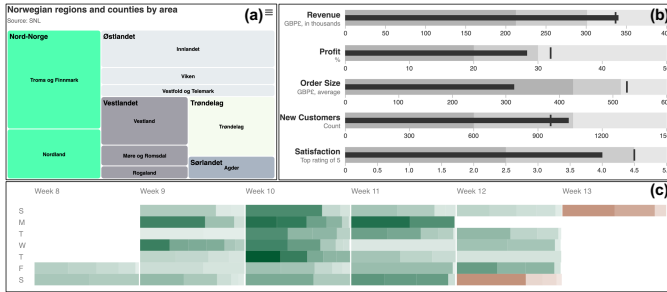


Fig. 7: Three new kinds of failure cases have been observed when evaluating the hierarchical clustering algorithm in Mystique [15] on a validation chart set generated from VISANATOMY: (a) a treemap visualization, (b) a bullet chart, and (c) a bespoke Gantt-calendar chart.

In summary, the generalizability of Mystique’s hierarchical clustering algorithm is satisfactory, which corroborates the authors’ claim on their corpus’ diversity. Nevertheless, the new error cases and Mystique’s lack of support for additional mark types and layouts call for more intelligent decomposition and mixed-initiative reuse approaches.

4.3 Chart Type Classification

Chart type classification is a typical visualization task, serving as the first step in many end-to-end systems such as Revision [66],

ChartSense [39], and REV [61]. Various models, including Support Vector Machines (SVMs) [20] and Convolutional Neural Networks (CNNs) [45], have been explored. However, existing work mostly assumes bitmap images as inputs, and focuses on coarse chart taxonomies with approximately a dozen categories. Considering that SVG is a highly structured data format, Graph Neural Networks [67, 76] could be a good fit [16], which have shown better results than image feature-only models in tasks such as chart retrieval [46] on a corpus of Plotly [2] charts. In this case study, we explore two questions: 1) how well GNNs classify SVG charts annotated with various semantic labels, and 2) how vision models and graph models perform when the number of chart types increases to 40.

Node Feature Extraction and Inclusion Criteria. If we focus on the raw SVG representations, we can designate basic shape elements such as `<line>` and `<circle>` as nodes in the graph. These elements correspond to **All Graphic Primitives** in VISANATOMY, and contain **main chart marks**, **Reference Elements**, and background noise (e.g., background rectangles, offscreen tooltips). To extract node-level features, we consider the following common features shared by different SVG element types for simplicity: (1) node-type, which is a one-hot encoding over the SVG element types, (2) node-position, which is a four-dimensional vector indicating the node’s bounding box in top, right, bottom, and left coordinates, and (3) node-style, a three-dimensional binary vector indicating the existence of the fill, stroke, and stroke-width style properties in the SVG file. We can scale the node-position feature using the width and height of the source SVG chart so that all values are within the range [0, 1]. In addition to shape elements, we also designate SVG container elements like `<g>` and `<SVG>` as graph nodes, which only have the node-type feature with other feature dimensions filled with 0.

With the semantic labels in VISANATOMY, we can also restrict graph nodes to elements with **Main Chart Mark** as their **Element Role**, so we only focus on the chart content and remove potential noise. In addition, the node-type feature from the raw SVG file can be inaccurate, as it is normal for SVGs to represent different shapes (rectangles, circles, etc.) with `<path>` elements [16]. Thus, a more accurate version of the node-type feature can be the one-hot encoding over the ground-truth mark types from **Element Type**.

Edges Definitions. Once the graph nodes are processed, we consider two potential ways to define edges. First, we can add edges based on the hierarchical organization between SVG elements and their parent containers in the raw SVG file. Alternatively, we can add the groups from **Hierarchical Grouping** (instead of the SVG container elements) as nodes to the graph, and construct edges based on the relation of subordination in **Hierarchical Grouping**.

Graph Construction. Combining the above variations of node definition, feature extraction, and edge definition, we present the following four graph representations with increasing amounts of semantic labels:

- SVG-Only (Graph 1): SVG shape and container elements as nodes, and parent-child relationships from the SVG hierarchy as edges;
- SVG-MainChart (Graph 2): main chart marks based on **Element Role** and SVG container elements as nodes, and parent-child relationships from the SVG hierarchy as edges;
- SVG-MainChart-MarkType (Graph 3): main chart marks based on **Element Role** and SVG container elements as nodes, with ground-truth one-hot encoding of **Element Type** as the node-type feature, and parent-child relationships from the SVG hierarchy as edges;
- SVG-MainChart-MarkType-Grouping (Graph 4): main chart marks based on **Element Role** and groups from **Hierarchical Grouping** as nodes, with ground-truth one-hot encoding of **Element Type** as the node-type feature, and group-child relationships from **Hierarchical Grouping** as edges.

Tasks. We consider two classification tasks: a 6-category problem where we follow the taxonomy from VisImage [27] to generate 6 high-level categories (Area, Bar, Circle, Line, Point, Grid&Matrix) covering 28 chart types, and the full 40-type problem. Within each type, we split charts into the train and test sets randomly using a 6 : 4 ratio.

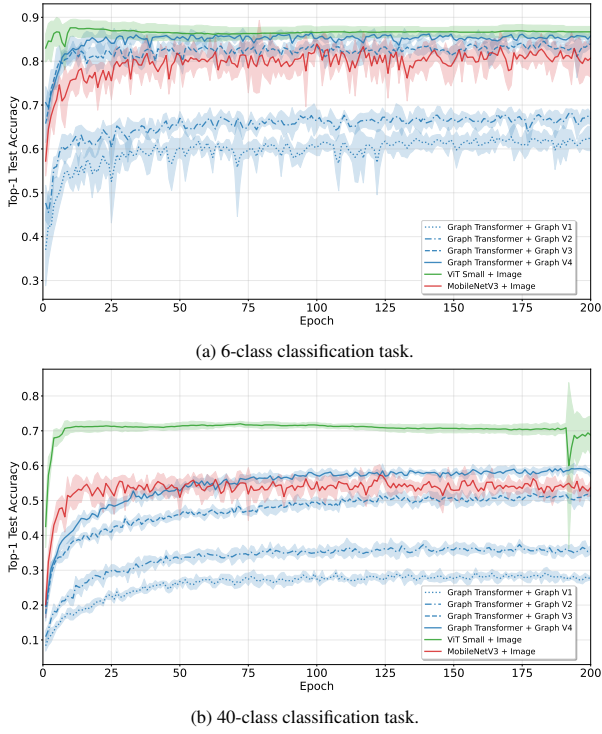


Fig. 8: Learning curves from the six models: top-1 test accuracy.

Model Training. We use a 3-layer Graph Transformer [77] network with 1.37M parameters to perform graph-based inference with Graph 1 to Graph 4. We also finetune on two pretrained vision models: MobileNet-v3-small (2.55M parameters) [35] and ViT-small (22.1M parameters) [13] with the bitmap images in VISANATOMY. For each model, we perform 5 independent runs to minimize noises; during training, we use the Adam optimizer [42] with a learning rate of 0.001 for graph models and 0.0001 for vision models for 200 epochs. The machine was NVIDIA GeForce RTX 2080Ti.

Results. Figure 8 presents the learning curves from the six models regarding the top-1 test accuracy. Generally, in both tasks we can observe that with more semantic information revealed to Graph Transformer, its performance is significantly enhanced (especially from Graph 3 to Graph 4), demonstrating the usefulness of **Element Role**, **Element Type**, and **Hierarchical Grouping** from VISANATOMY.

Compared to MobileNet-v3 which has 86% more parameters, Graph Transformer obtains the same-level performance with Graph 3 and better test accuracy with Graph 4. Although semantic labels are required prior to training, the Graph Transformer can be trained much faster: its average second-per-epoch is 0.53 while that for MobileNet-v3 is 30.8. The best performance is given by ViT-small, which has approximately 16.7% gain in test accuracy compared to Graph Transformer with Graph 4 for the 40-class task. However, ViT-small is a much larger model with 15 times more parameters and an average second-per-epoch of 33.37. Based on the result, test-time label inference and semantic-rich graph construction for enhanced graph-based modeling are promising directions for future work.

4.4 Content Navigation for Accessibility

VISANATOMY not only serves as a benchmark dataset for evaluating algorithms and models, but also as a valuable resource for researchers to develop visualization applications that use SVG charts as input. We demonstrate its utility by replicating a project focused on chart accessibility, which involves designing chart reading experiences for people with visual impairments. Current visualization accessibility practices link textual descriptions of charts (usually provided by the authors through alt texts) to the underlying data tables so that assistive screen readers can communicate some high-level chart semantics to

the user [73]. However, this paradigm does not offer visually impaired individuals the same level of chart exploration experiences that sighted people can access through interactive visualizations [79]. In response, Zong et al. [79] propose a chart accessibility tree as the underlying structure for traversal of a chart’s scene graph: the user navigates along multi-level branches of the accessibility tree through a keyboard.

Considering that the demos provided by Zong et al. [79] are implemented with Vega-Lite charts, here we demonstrate that the semantic labels in VISANATOMY can support the construction of the accessibility trees for charts created using other tools. To re-create the accessible chart reading experiences, we focus on the five examples in the gallery of Zong et al.’s work (Figure 2 in [79]), and choose five charts from VISANATOMY that have very close visualization designs: a faceted connected dot plot [52] from Mascot.js, a multi-line chart [33] from HighCharts, a geographical heatmap [23] from D3.js, a stacked bar chart [7] from Apexcharts.js, and a bar chart with annotations [26] from *poetrybetweenpain.deb.is*. In our implementation, we focus on the following navigation patterns: structural navigation with the **Up**, **Down**, **Left**, **Right** arrow keys, spatial navigation with the **WASD** keys across grids, and lateral navigation across facets with the **Shift+Left** and **Shift+Right** key combinations. We next briefly introduce how we implemented the accessibility trees (illustrated in Figure 9) for the faceted connected dot plot and the multi-line chart example. We include the construction processes for the other three charts in the supplementary materials.

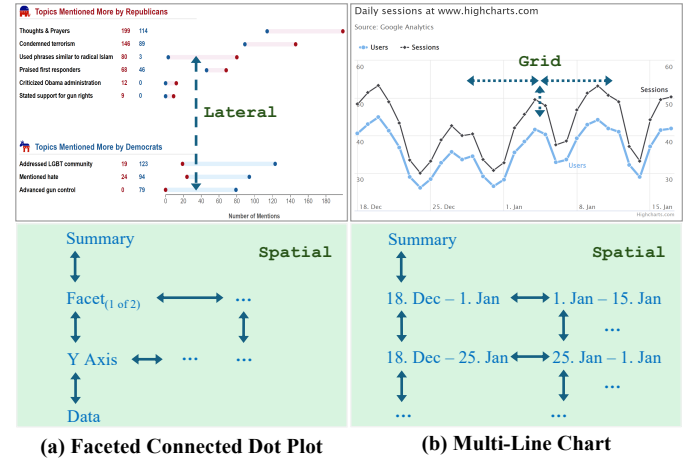


Fig. 9: Navigation patterns re-created in (a) a faceted connected dot plot and (b) a multi-line chart using labels from VISANATOMY.

Faceted Connected Dot Plot (Figure 9(a)): Starting with the whole SVG chart as the root, we introduce two branches corresponding to the two facets using **Hierarchical Grouping**. Each branch has several connected dot pairs as children, which are linked to their corresponding y-axis labels in **Reference Elements** based on their y coordinates obtained from **All Graphic Primitives**. Once the **Shift + Left/Right** combination is detected, the focus will be shifted to the other facet branch’s dot pair of the same index to support the lateral navigation. The leaf nodes in the structural navigation tree are the individual dots.

Multi-Line Chart (Figure 9(b)): A binary accessibility tree similar to that in [79] is formed. The range of the x-axis labels from **Reference Elements** is split into two, each is attached to the SVG root node as a child and linked with marks whose x coordinates are in the range. This binary range partition continues until only one mark is inside; the whole binary accessibility tree is then used for the structural navigation. We further support spatial navigation with the **WASD** keys across grids: x and y coordinates of gridlines from **Reference Elements** are obtained to cut the coordinate system into 2-dimensional grids, which are regarded as children of the SVG root. The **S** key triggers the spatial navigation mode starting with the upper-left grid and the **Up** arrow key brings the user back to the root.

Overall, we find it straightforward to construct the chart accessibility tree using the semantic labels from VISANATOMY: once the tree structure is decided, the mappings between the tree nodes and the graphical objects in the chart can be retrieved with the help of [Hierarchical Grouping](#), [Element Type and Role](#), position and color properties in [All Graphic Primitives](#), and axis and legend information from [Reference Elements](#). The supplementary materials include demo videos and a web application for re-created navigation patterns on the five charts. However, we have also observed a few places where semantics beyond VISANATOMY are required. For example, for the geographical heatmap example, the underlying CSV data from the source website is needed to obtain names of provinces and cities represented by marks; for the annotated bar chart example, the affixation of annotation texts onto the main chart bars needs to be determined in advance. We include more discussion on these issues in Section 5.

5 DISCUSSION AND FUTURE WORK

Dealing with Real-World SVG Charts. SVG charts found in the wild exhibit considerable noise and heterogeneity, even among charts of the same type. The semantic labels in VISANATOMY significantly reduce the noise, but there are still edge cases we cannot handle properly (e.g., the entire box glyph in a box-and-whiskers plot drawn using a single `<path>` element). The semantic labels for the same type of charts can also vary. For instance, in some bullet charts, the rectangles in the same glyph are overlapping and aligned to one side (e.g., left or bottom), while others stack the rectangles without overlapping (Figure 7(b)). We label the former as a group with a glyph layout, and the latter as a group with a stack layout. The implications of these cases on downstream applications remain to be explored and better understood.

Strengthening Data Component Labeling. Currently VisAnatomy has two kinds of labels recording mappings between data and visuals: 1) [Visual Encodings](#) record which visual element (e.g., mark, collection) and which channels of that element (e.g., color, position) encode data, and 2) [Reference Elements](#) record the type of data attribute (e.g., number, string) and the channel (e.g., width, x position) for each axis. For mappings that do not have associated axes or legends, information on the type of data attribute is missing and needs to be included.

In its current state, only 392 charts (out of 942) in VISANATOMY have the underlying data tables available. However, data tables offer important information such as data schema and attribute values that can be useful for applications like visualization redesign and recommendation [37]. In future work, we plan to augment charts that lack underlying data tables by investigating methods to automatically reconstruct these tables using existing labels such as [Reference Elements](#) and [Visual Encodings](#). If fully automated approaches are not possible, we plan to extend the current data extraction methods (e.g., ChartDetective [53]) and add a **Data Table** stage in our labeling tool, allowing human-machine collaborative curation of the underlying data table.

Labeling Inter-Element Relationships as Constraints. Some chart scene abstraction frameworks, such as Charticulator [62] and MSC [48] which inspired the semantic labels in VISANATOMY (Section 3.1), have a **constraint** component that describes the spatial relationships between elements. Examples include *align* (e.g., customized alignments in stacked bar charts) and *affix* (e.g., the relative positioning between a mark and its annotations). VISANATOMY currently does not have labels on such constraints. The main challenge is that manually linking every pair of elements (e.g., a bar and its text annotation) and specifying their relationships as constraints can be time-consuming and error-prone. We need automatic algorithms to recognize and predict such constraints in batch, and novel interaction models to support generalizable constraint labeling.

AI-Assisted Labeling. A significant portion of our time on this project was dedicated to developing and refining the labeling system. Labeling one chart using the system takes 10 to 15 minutes on average right now. The labeling efficiency can be further improved with the incorporation of AI models. In Section 4.1, we have shown that current LLMs can predict the semantic roles of SVG elements with good accuracy overall, such labels can thus be automatically populated. However, we

need better interfaces for humans to verify and correct mistakes, especially for axis labels and main chart marks. We also plan to investigate additional AI support (e.g., fine-tuning LLMs using VISANATOMY) for labeling other components like grouping and encoding.

Further Enhancing the Corpus. Although the number of charts in VISANATOMY is comparable to state-of-the-art chart corpora curated using manual approaches (Section 3.4), and the scale of fine-grained labels can support various applications (Section 4), VISANATOMY can be enlarged further with more chart designs and intra-type variations. We will open-source VISANATOMY and our labeling tool to encourage contributions from the visualization community. We have also experimented using synthetic data to augment the size of VISANATOMY. To do this, we (1) converted charts in VISANATOMY to scene templates in Mascot.js [48], (2) generated compatible synthetic datasets, and (3) used Mascot.js to infuse the synthetic datasets with the templates. Example charts are included in the supplementary materials. We decide not to include these generated charts in VISANATOMY: the distribution of the synthetically generated data should be determined according to the specific machine-learning task or interactive application; it is better that VISANATOMY only contains the original real-world charts, which can be augmented in different ways depending on the use case. Future research may develop approaches to promoting diversity in the underlying data and the visual styles at the same time, and to achieve a desired balance between quantity and diversity.

VISANATOMY does not yet include node-link visualizations. The main challenge in labeling is similar to that in labeling constraints: automatic algorithms are needed to recognize and predict relationships between nodes and links. Composite visualizations, where one chart overlays on another with each chart having its own axis, are not within the scope either. By focusing on SVG charts, we may also have omitted unique visualization designs that are available only in formats like raster images. VISANATOMY can be further enriched by including these missing charts, with AI-assisted methods to label components.

Enhancing the Breadth and Depth of Applications. The GNN models tested in Section 4.3 are homogeneous, having the same feature length for all the nodes. Supporting node features of unequal lengths for different types of nodes could be a better approach. Second, the semantic labels have the potential to support the development of rigorous approaches to computing pair-wise chart similarities, which can serve as a quantitative measure to evaluate corpus diversity [16], guiding the future collection of chart corpora. Third, for research that combines Visualization and Natural Language (NL) to solve tasks such as chart QA [40] and chart captioning [51], the diverse set of charts in VISANATOMY and the associated semantic labels can be utilized to synthesize datasets that contain high-quality (VIS, NL) pairs, enhancing the generalizability of multi-modal models.

Supporting Interaction and Animation Labeling. Currently VISANATOMY focuses on semantic labels in static charts. A future direction is to annotate interaction and animation. The semantic labels of static components are based on existing visualization abstraction models; we envision that the annotation of interaction and animation also requires solid theoretical foundations on dynamic visualizations. We will investigate how interaction grammars like Vega-Lite [65] may be used and explore additional abstractions.

6 CONCLUSION

In the paper, we contribute VISANATOMY, a diverse SVG chart corpus encompassing 40 chart types produced by over 50 tools from hundreds of public online sources. Each chart in VISANATOMY is augmented with rich, multi-granular semantic labels including graphical elements’ types, roles, and bounding boxes, the hierarchical grouping of elements, the layouts of groups, and visual encodings. We compare VISANATOMY with related corpora to demonstrate its diversity and the richness of the semantic labels. The usefulness of VISANATOMY is evaluated through four applications: semantic inference of SVG element roles, chart semantic decomposition, chart type classification, and content navigation for accessibility. Finally, we outline research challenges and opportunities for future work.